

TRABALHO 1 EDA EXERCÍCIOS,

RODRIGO DUTRA

FERREIRA,242015951

```
#include <stdio.h>

//Exercício 1 – Inspeção de memória
//Declare:
// uma variável inteira
// uma variável float
// um ponteiro para cada tipo
// Atribua corretamente os endereços
// Imprima:
// valor da variável

int main(){
    int numero = 100;
    float numero2= 200;
    int *ptr1 = &numero;
    float *ptr2 = &numero2;
    printf(" Numero1 = %d, Numero2 = %.2f", *ptr1, *ptr2);
}

#include <stdio.h>

//Exercício 2 – Ponteiros cruzados
// p1 aponta para A, p2 aponta para B
// Imprima os valores apontados
// Troque os ponteiros (sem trocar os valores das variáveis)
// Imprima novamente

void TrocarNumeros( float *p1, float *p2){
    float temp = *p1;
    *p1 = *p2;
    *p2 = temp;
}

int main(){
    float a = 10;
    float b= 20;
    printf("Numero a= %.2f , e b= %.2f \n", a, b);

    TrocarNumeros(&a, &b);
}
```

```

    printf("Numeros trocados a= %.2f , e b= %.2f \n", a, b);

}

#include <stdio.h>
// Exercício 3 – Incremento indireto
// Declare uma variável inteira
// Use um ponteiro para:
// somar 5
// multiplicar por 2
// Imprima o resultado final
// Atenção: Não acessar diretamente a variável (somente via ponteiro).

int main(){
    int variavel = 100;
    int *p1 = &variavel;
    *p1 = *p1 + 5;
    *p1 = *p1 * 2;
    printf("Incremento direto: variavel = %d", *p1);
}

#include <stdio.h>
// Exercício 4 – Swap manual (sem função)
// Declare duas variáveis inteiras
// Use ponteiros para trocar os valores dentro da main
// Não use variável auxiliar tradicional (use manipulação via ponteiros)

int main(){
    int numero1 = 10;
    int numero2 = 20;
    int *ptr1 = &numero1;
    int *ptr2 = &numero2;
    int temp = *ptr1;
    *ptr1 = *ptr2;
    *ptr2 = temp;
    printf("numero1: %d, numero2: %d", *ptr1, *ptr2);
}

#include <stdio.h>
// Exercício 5 – Vetor com ponteiros
// Dado um vetor:
// int v[5] = {10, 20, 30, 40, 50};
// Faça:
// Percorra usando ponteiro (não use índice)
// Imprima os valores
// Some todos os elementos usando apenas aritmética de ponteiros
// Dica: use *(p + i) ou incremento de ponteiro

```

```

int somaVetorPonteiros(int *v){
    int somador = 0;
    for (int i= 0; i < 5; i++){
        somador += *(v + i); // ATENÇÃO

    }
    return somador;
}

int main(){
    int vetor1[] = {10, 20, 30, 40, 50};
    int resultado = somaVetorPonteiros(vetor1);
    printf("Resultado da soma dos vetores: %d", resultado);
}

#include <stdio.h>
// Exercício 6 – Modificação por função
// Implemente uma função:
// void dobra(int *x);
// A função deve dobrar o valor de x
// Teste na main
// Verifique: o que acontece se passar sem ponteiro?

void dobrarNumero(int *x){
    *x = *x * 2;
}

int main(){
    int numero1 = 10;
    int *ptr1 = &numero1;
    dobrarNumero(ptr1);
    printf("Numero1: %d", *ptr1);
}

#include <stdio.h>
// Exercício 7 – Maior elemento do vetor
// Implemente:
// int maior(int *v, int tamanho);
// Use apenas ponteiros (sem v[i])
// Retorne o maior valor

int maior(int *v, int tamanho){
    int max = *v;
    for (int i= 0; i < tamanho; i++){
        if ( *(v + i) > max){

```

```

        max = *(v + i);
    }

}

return max;
}

int main(){
    int vetor1[] = {1, 2, 3, 4};
    int *ptr = vetor1;
    int resultado = maior(ptr, 4);
    printf("Maior : %d", resultado);
}

#include <stdio.h>
#include <string.h>
//Exercicio 8
// Dada uma string:
// char nome[] = "programacao";
// Crie uma função:
// int conta(char *str, char c);
// Retorne quantas vezes c aparece
// Use apenas ponteiros

int conta( char *str, char c){
    int contagem = 0;
    while (*str != '\0'){
        if (*str == c){
            contagem++;
        }
        str++; //avança ponteiro
    }
    return contagem;
}

int main(){
    char nome[] = "programacao";
    char c = 'a';
    int resultado = conta(nome, c);
    printf("o char '%c' aparece %d vezes", c, resultado);
}

#include <stdio.h>
// Exercício 9 – Inversão de vetor (in-place)
// Crie uma função:

```

```

// void inverte(int *v, int tamanho);
// Inverta os elementos do vetor
// Não use vetor auxiliar
// Use dois ponteiros (início e fim)
// Exemplo: [1, 2, 3, 4] → [4, 3, 2, 1]

void inverte(int *v, int tamanho){
    int *inicio = v;
    int *final = v + (tamanho - 1);
    int temp;
    while ( inicio < final){
        temp = *inicio;
        *inicio = *final;
        *final = temp;
        inicio++;
        final--;
    }
}

int main(){
    int vetor[] = {1, 2, 3, 4};
    int tamanho = 4;
    inverte(vetor, 4);
    printf("Vetor invertida:");
    for (int i = 0; i < tamanho; i++) {
        printf("%d ", vetor[i]);
    }

    return 0;
}

#include <stdio.h>
// Exercício 10 – Intercalação de vetores
// Crie uma função:
// void intercala(int *v1, int *v2, int *v3, int n);
// Intercale os elementos de v1 e v2 em v3
// Resultado: v3 com tamanho 2n
// Regra: usar apenas aritmética de ponteiros

void intercala(int *v1, int *v2, int *v3, int n){
    for (int i = 0; i < n; i++) {

        *(v3 + (2 * i)) = *(v1 + i);
    }
}

```

```

        *(v3 + (2 * i + 1)) = *(v2 + i);
    }
}

int main(){
    int vetor1[] = {1, 2};
    int vetor2[] = {3, 4};
    int vetor3[4];
    int n = 2;
    intercala( vetor1, vetor2, vetor3, n);
    printf("Vetor intercalado:");
    for (int j = 0; j < 2 * n; j++) {
        printf("%d ", *(vetor3 + j));
    }

    return 0;
}

#include <stdio.h>
// Exercício 11 – Comparação de strings
// Implemente sua própria versão de:
// int compara(char *s1, char *s2);
// Retorne:
// 0 se iguais
// <0 se s1 < s2
// 0 se s1 > s2
// Sem usar strcmp
int compara(char *s1, char *s2) {
    while (*s1 == *s2 && *s1 != '\0') {
        s1++;
        s2++;
    }
    return (unsigned char)*s1 - (unsigned char)*s2;
}

int main() {
    char string1[] = "arvore";
    char string2[] = "galho";

    int resultado = compara(string1, string2);

    if (resultado == 0) {
        printf("As strings sao iguais.\n");
    } else if (resultado < 0) {
        printf("A string1 vem antes da string2 (s1 < s2).\n");
    }
}

```

```
} else {  
    printf("A string1 vem depois da string2 (s1 > s2).\n");  
}  
  
printf("Valor do retorno: %d\n", resultado);  
  
return 0;  
}
```